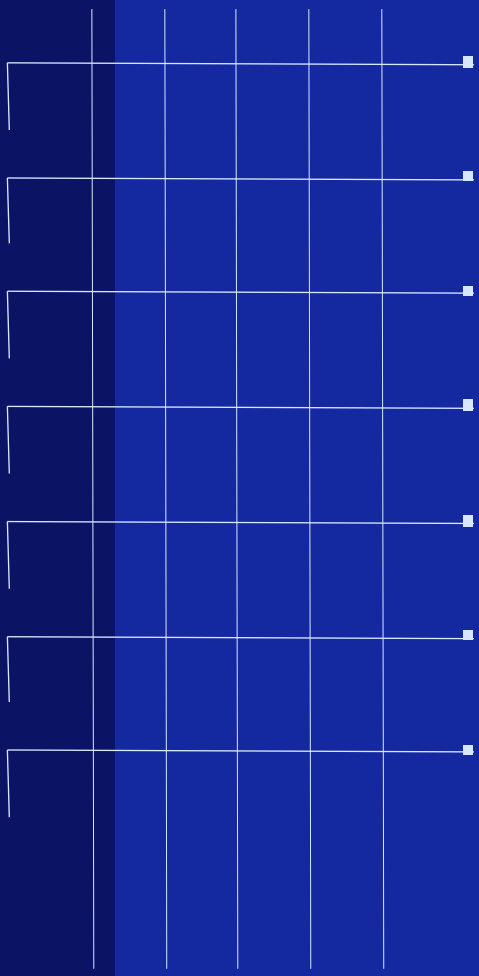
Samsung Innovation Campus

GRADUATION PROJECTS HANDBOOK

Big Data Engineering Track



SIC 8 | 2026

An end-to-end working solution

Choose one domain and deliver a complete technical system that connects a real business problem to tested, demonstrable outputs.

01

Problem Definition

Translate the domain challenge into measurable technical objectives and a clear stakeholder use case.

02

Solution Architecture

Define the data flow, system components, interfaces and selected deployment pathway.

03

Technical Development

Implement the generator, ingestion, processing, lakehouse and BI workflow.

04

Testing & Validation

Check data quality, failure handling, analytical correctness and pipeline behavior.

05

Final Presentation

Demonstrate the working system, explain results and limitations, and substantiate each contribution.

Learning outcomes: demonstrate reproducible engineering, validate technical decisions and communicate evidence-based business insights.

Required = core scope / Recommended = implementation guidance / Optional = extension

Five domains. One selected project.

01



Banking, FinTech & Payment Systems

Real-Time Payment Ingestion & Anti-Fraud Lakehouse

Payment events, suspicious activity and terminal reliability.

02



Smart Healthcare & Clinical Telemetry (IoMT)

Scalable IoMT Vitals Ingestion & Clinical Analytical Warehouse

High-frequency simulated vital signs and sensor disruptions.

03



Supply Chain & Intelligent Logistics

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Fleet operations, route efficiency and cold-chain monitoring.

04



Smart Energy, Power Grids & Utilities

High-Scale Smart Meter (AMI) Ingestion & Cloud/Local Data Platform

Large meter populations, periodic readings and grid-demand analysis.

05



Retail, E-Commerce & Customer Journey

Real-Time Clickstream Ingestion & Omnichannel Data Warehouse

Web/mobile journeys, abandoned carts and marketing performance.

The five engineering pillars

Required for every project, on either infrastructure pathway.

01 Domain-Specific Data Generation

Build custom Python event generators with realistic business schemas using JSON, Avro or CSV. Use concurrent generation through threading or Asyncio as appropriate, with tools such as Faker. Inject controlled domain anomalies.

Support real-time broker streaming and scheduled bulk micro-batches. Keep generator settings and anomaly controls reproducible. A fixed CSV alone does not satisfy the generator requirement.

02 Data Ingestion

Use Kafka or a managed equivalent for high-velocity topics. Use NiFi dataflows or scheduled extractor scripts for periodic landing. Preserve ingested data in an immutable bronze/raw zone on S3, ADLS, GCS, MinIO or HDFS.

Implement dead-letter queues, ingestion logs and retry backoff. Retain evidence that malformed or failed events are visible and recoverable.

03 Distributed Processing

Implement scalable PySpark batch jobs or Structured Streaming. Enforce schemas, handle nulls, deduplicate composite business keys and enrich events through lookups. Use Snappy-compressed Parquet or ORC with physical partitions.

Control shuffles. For RDDs, prefer reduceByKey/aggregateByKey to groupByKey where applicable. Broadcast small dimensions and use window functions for rolling aggregates rather than repeated full scans.

Validation, serving & architecture

04 Lakehouse & Dimensional Warehousing

Implement Iceberg or Delta Lake with ACID transactions, schema enforcement, time-travel queries, updates/merges and partition pruning. Handle slowly changing dimensions: Type 1 overwrites values, while Type 2 retains history. Build a star or snowflake schema with surrogate dimension keys and explicit fact grain.

Identify additive, semi-additive and non-additive metrics. Expose tables through Trino, Presto, HiveQL on Tez or cloud OLAP. Apply pre-commit quality gates using Great Expectations or custom Spark assertions.

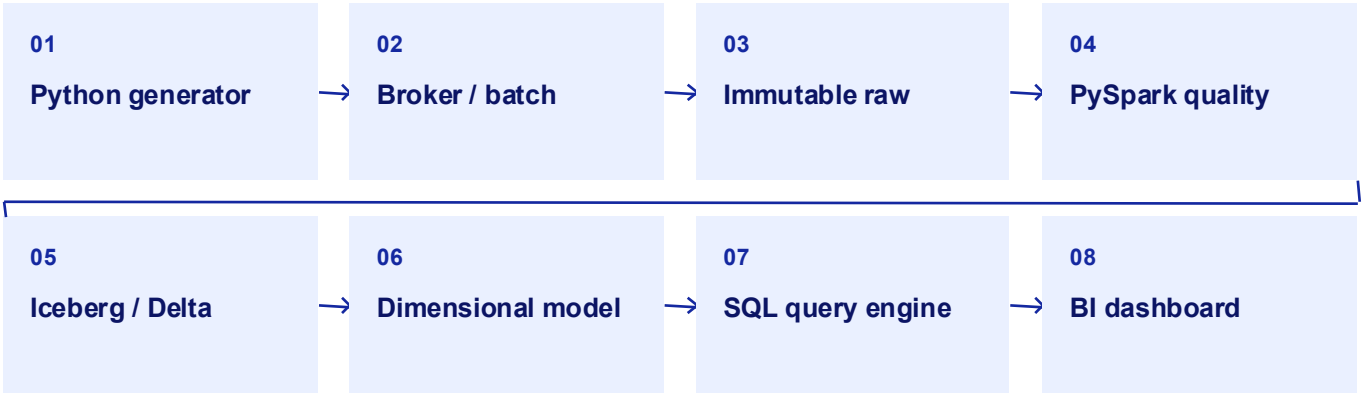
05 Orchestration & Analytics

Automate end-to-end dependencies using Apache Airflow DAGs. Connect curated data to interactive Superset, Power BI, Metabase or equivalent dashboards with domain-specific operational KPIs.

Demonstrate the complete execution path and retain workflow logs, processing results and quality evidence. Distinguish measured performance from assumed scale.

Airflow coordinates pipeline dependencies from ingestion to analytical serving.

END-TO-END ARCHITECTURE



Two equally valid pathways

Choose the environment that supports your project requirements and available resources. Local implementation carries no disadvantage solely because it avoids paid cloud services.

Architectural tier	A Cloud-based	B Local / on-premises
Storage / raw lake	S3, ADLS Gen2 or Google Cloud Storage	MinIO or HDFS
Ingestion	MSK, Kinesis, Event Hubs or Pub/Sub	Kafka, NiFi or scheduled extractors
Distributed compute	EMR, Databricks or Dataproc Spark	Standalone PySpark cluster, Docker or native Linux
Lakehouse / warehouse	Iceberg / Delta with Athena, Snowflake or BigQuery	Iceberg / Delta on MinIO/HDFS with Trino or Hive
Orchestration	Managed Airflow, such as MWAA	Apache Airflow with Docker Compose
Analytics / BI	Power BI, Preset or equivalent	Superset, Metabase or equivalent

Assessment follows functionality, implementation quality, reliability and reproducibility. Infrastructure cost does not earn marks.

Banking, FinTech & Payment Systems

Real-Time Payment Ingestion & Anti-Fraud Lakehouse

A BUSINESS CHALLENGE

Payment operations and fraud analysts need a coherent view of rapid transaction activity across channels. Raw events must survive ingestion failures and become reliable, queryable evidence for investigating anomalies.

B PROJECT OBJECTIVE

Build a simulated payment platform that ingests transactions, preserves raw events and serves fraud and operational KPIs through a lakehouse and dashboard.

C SCOPE OF WORK

Connect a multi-threaded Python generator to ingestion, immutable raw storage, PySpark transformations, transactional lakehouse tables, dimensional analytics and an Airflow workflow.

D TECHNICAL REQUIREMENTS

Generator & anomalies	Simulate global POS swipes, e-commerce checkouts and ATM withdrawals. Inject rapid card-not-present bursts, geographic velocity violations and micro-testing scams. The source example uses transactions 5,000 miles apart within 15 minutes.
Processing & serving	Preserve transaction/account/terminal context. Compute sliding 10-minute transaction velocity per account, real-time fraud detection rate and terminal failure rates. Explain how flags relate to injected events.
Infrastructure options	Cloud example: S3, MSK, PySpark on EMR/Databricks and Athena. Local example: Dockerized Kafka, MinIO/HDFS, local PySpark and Trino. Add the shared lakehouse, quality and orchestration requirements.

Real-Time Payment Ingestion & Anti-Fraud Lakehouse

01 **Payment simulator** → 02 **Broker + raw lake** → 03 **Spark + lakehouse** → 04 **Fraud/terminal KPIs**

E EXPECTED FUNCTIONALITIES

01 Generate and ingest payment activity with all specified anomaly types.

02 Query account velocity over sliding 10-minute windows.

03 Refresh fraud and terminal-failure indicators from processed data.

F MINIMUM ACCEPTANCE CRITERIA

- ☐ The generator produces all three transaction channels and anomaly families.
- ☐ Injected cases can be traced from raw landing through curated outputs.
- ☐ The three specified KPIs are queryable and visible in the dashboard.
- ☐ The shared five-pillar pipeline, validation gates and Airflow execution are demonstrated.

G EXPECTED DELIVERABLES

Payment generator/configuration, event schema and anomaly definitions, ingestion configuration, Spark jobs, lakehouse tables and dimensional model, KPI SQL/dashboard and execution evidence.

IMPLEMENTATION RECOMMENDATIONS

Define the numerator and denominator of fraud detection rate against synthetic truth. Keep anomaly labels out of detection rules that would reveal the answer. Document window boundaries, currency treatment and terminal outcome semantics.

H ADVANCED ENHANCEMENTS (OPTIONAL)

Stress-test burst traffic, add replay diagnostics or compare detection rules under different synthetic fraud mixes.

Smart Healthcare & Clinical Telemetry (IoMT)

Scalable IoMT Vitals Ingestion & Clinical Analytical Warehouse

A BUSINESS CHALLENGE

Clinical operations teams need a consistent view of telemetry from many bedside and wearable devices. Physiological variation and disconnected sensors require distinct handling before data can support analysis.

B PROJECT OBJECTIVE

Build a scalable simulated IoMT ingestion and analytical warehouse that preserves vital-sign events and makes emergency patterns and sensor issues visible.

C SCOPE OF WORK

Connect a Python device simulation to ingestion, immutable raw storage, PySpark cleaning and enrichment, transactional tables, a clinical dimensional warehouse and an orchestrated dashboard.

D TECHNICAL REQUIREMENTS

Generator & anomalies	Represent hundreds of ICU bedside monitors and wearable units. Generate heart rate (BPM), SpO2, blood pressure and body temperature with physiological noise. Inject acute tachycardia, rapid desaturation and sensor disconnects as null readings.
Processing & serving	Apply schema validation, null handling and deduplication while preserving evidence of disconnections. Relate readings to devices and the clinical simulation context. Serve operational analysis of the generated vitals and anomalies.
Infrastructure options	Cloud example: Azure Event Hubs, ADLS Gen2 and Synapse/Databricks Spark. Local example: Apache NiFi, MinIO/HDFS, PySpark and Trino/Hive on Tez. Apply all shared lakehouse and Airflow requirements.

Scalable IoMT Vitals Ingestion & Clinical Analytical Warehouse

01 **ICU/wearable simulator** → 02 **Ingestion + raw lake** → 03 **Cleaned vitals warehouse** → 04 **Clinical telemetry view**

E EXPECTED FUNCTIONALITIES

01 Produce and ingest continuous readings from the simulated device population.

02 Distinguish physiological events from disconnected-sensor readings.

03 Query vital-sign trends and display operational telemetry analytics.

F MINIMUM ACCEPTANCE CRITERIA

- ☐ The configured simulation covers hundreds of units and all four vital-sign types.
- ☐ Tachycardia, desaturation and null-reading cases appear in evidence.
- ☐ Raw, cleaned and analytical layers preserve traceable device/time context.
- ☐ Quality checks, orchestration and the working dashboard execute end to end.

G EXPECTED DELIVERABLES

Device simulator and configuration, vital-sign schema/units, anomaly scenarios, ingestion and Spark code, lakehouse/warehouse definitions, analytical queries and dashboard evidence.

IMPLEMENTATION RECOMMENDATIONS

Useful dashboard measures include event counts, missing-reading rate and abnormal-event trends. Define them explicitly. Keep the system a synthetic educational simulation and avoid claims of clinical readiness or medical diagnosis.

H ADVANCED ENHANCEMENTS (OPTIONAL)

Event-time lateness analysis, device-health monitoring or configurable physiological scenario playback.

Supply Chain & Intelligent Logistics

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

A BUSINESS CHALLENGE

Fleet managers need to combine vehicle diagnostics, position and cargo conditions. Disconnected telemetry streams make route deviations, refrigeration breaches and excessive idling difficult to investigate consistently.

B PROJECT OBJECTIVE

Build a telematics lakehouse and dimensional warehouse for a simulation of 5,000 delivery vehicles on fixed logistics corridors.

C SCOPE OF WORK

Generate continuous vehicle telemetry, ingest and retain raw data, enrich events through PySpark and serve route, cold-chain and driver analytics through an orchestrated data platform.

D TECHNICAL REQUIREMENTS

Generator & anomalies	Track 5,000 vehicles on fixed corridors. Generate GPS latitude/longitude, OBD-II fuel consumption, RPM and engine temperature, plus cargo refrigeration temperature. Inject cold-chain breaches, excessive idling and route deviations.
Processing & serving	Preserve vehicle, route and time relationships. Deliver fuel efficiency per route, cold-chain compliance percentage, and driver safety/idling scorecards with documented metric definitions.
Infrastructure options	Cloud example: S3, Kinesis, PySpark on EMR and Snowflake. Local example: Dockerized Kafka, HDFS/MinIO, PySpark and Trino. Retain transactional lakehouse, SCD, quality-gate and Airflow components.

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

01 **5,000-vehicle simulator** → 02 **Telemetry ingestion** → 03 **Lakehouse + dimensions** → 04 **Route/fleet scorecards**

E EXPECTED FUNCTIONALITIES

01 Generate the required fleet telemetry and anomaly scenarios.

02 Query route-level fuel efficiency and refrigeration breaches.

03 Display driver safety and idling scorecards from curated data.

F MINIMUM ACCEPTANCE CRITERIA

- ☐ Configuration and output evidence cover 5,000 distinct vehicles.
- ☐ All specified telemetry fields and three anomaly families are present.
- ☐ Required route, cold-chain and driver KPIs reconcile with analytical queries.
- ☐ A complete orchestrated run connects raw events to the warehouse and dashboard.

G EXPECTED DELIVERABLES

Fleet generator and corridor definitions, telemetry schema, ingestion/Spark jobs, lakehouse and warehouse models, scorecard logic, dashboard and performance/execution logs.

IMPLEMENTATION RECOMMENDATIONS

State units and threshold assumptions for each metric. Treat cold-chain compliance as adherence to the simulation threshold, without implying regulatory certification. Record achieved throughput and resource use on the chosen environment.

H ADVANCED ENHANCEMENTS (OPTIONAL)

Geospatial enrichment, partition and join optimization, or fleet playback for anomaly investigations.

Smart Energy, Power Grids & Utilities

High-Scale Smart Meter (AMI) Ingestion & Cloud/Local Data Platform

A BUSINESS CHALLENGE

Utility analysts must aggregate many meter readings while retaining visibility into local load surges and irregular consumption. A useful platform must support both reliable periodic ingestion and detailed anomaly investigation.

B PROJECT OBJECTIVE

Build a data platform for 100,000+ simulated AMI meters reporting active power, reactive power and voltage every 15 minutes.

C SCOPE OF WORK

Connect the high-scale meter generator to ingestion, immutable raw storage, distributed transformations, a transactional lakehouse and a dimensional serving layer for utility KPIs.

D TECHNICAL REQUIREMENTS

Generator & anomalies	Simulate at least 100,000 meters with the 15-minute reporting interval. Inject sudden transformer load surges, persistent line-drop anomalies and irregular consumption profiles that simulate power theft.
Processing & serving	Relate meters to the grid hierarchy and aggregate time-series readings. Serve substation transformer peak loading, consumption anomaly flags and daily aggregate grid demand, with consistent units and aggregation definitions.
Infrastructure options	Cloud example: GCS, Pub/Sub, Dataproc Spark and BigQuery. Local example: Apache NiFi, MinIO/HDFS, standalone PySpark and Hive on Tez/Trino. Apply shared lakehouse, quality and Airflow requirements.

High-Scale Smart Meter (AMI) Ingestion & Cloud/Local Data Platform

01 **100,000+ AMI meters** → 02 **15-minute readings** → 03 **Spark + lakehouse** → 04 **Grid demand + flags**

E EXPECTED FUNCTIONALITIES

01 Generate and ingest the required meter population and reporting cadence.

02 Expose transformer peak loading and daily grid-demand aggregates.

03 Flag irregular consumption and show underlying simulated readings.

F MINIMUM ACCEPTANCE CRITERIA

- ☐ Configuration and generated IDs demonstrate 100,000+ meters.
- ☐ Event timestamps represent 15-minute intervals with all specified measurements.
- ☐ All three anomaly families and specified KPIs appear in the demonstration.
- ☐ Raw-to-dashboard execution is automated, validated and reproducible.

G EXPECTED DELIVERABLES

AMI generator/configuration, meter/grid schema, anomaly scenarios, ingestion and Spark jobs, transactional/warehouse tables, KPI SQL, dashboard and scale/run evidence.

IMPLEMENTATION RECOMMENDATIONS

Document wall-clock runtime separately from simulated event time. Keep the required population and cadence in the data model. Explain whether a dashboard shows power or energy and how time aggregation works. Anomaly flags indicate synthetic patterns, not proof of actual theft.

H ADVANCED ENHANCEMENTS (OPTIONAL)

Skew/partition tuning, meter cohort analysis, historical replay or processing-cost profiling.

Retail, E-Commerce & Customer Journey

Real-Time Clickstream Ingestion & Omnichannel Data Warehouse

A BUSINESS CHALLENGE

Commerce analysts need to connect activity across web and mobile sessions. Missing or duplicated events can distort conversion funnels, while bot traffic and promotional surges change the shape of demand.

B PROJECT OBJECTIVE

Build a real-time clickstream lakehouse and omnichannel warehouse that converts simulated user journeys into trustworthy retail analytics.

C SCOPE OF WORK

Generate concurrent sessions, ingest events, preserve the raw stream, process journeys with Spark and serve dimensional analytics through an orchestrated dashboard.

D TECHNICAL REQUIREMENTS

Generator & anomalies	Simulate page views, search filters, cart updates, checkouts and exits across concurrent web/mobile sessions. Inject cart abandonment, bot-like crawling loops and seasonal promotional purchase surges.
Processing & serving	Produce funnel conversion drops, cart abandonment percentage, session dwell-time distributions and marketing channel ROI. Define session boundaries, funnel stages and the financial inputs used for ROI.
Infrastructure options	Cloud example: S3, MSK, Spark Structured Streaming and ClickHouse Cloud/Athena. Local example: Dockerized Kafka, MinIO, PySpark Streaming and Trino/local ClickHouse. Include shared lakehouse, quality and Airflow components.

Real-Time Clickstream Ingestion & Omnichannel Data Warehouse

01 **Web/mobile sessions** → 02 **Streaming ingestion** → 03 **Session + warehouse model** → 04 **Funnel/channel analytics**

E EXPECTED FUNCTIONALITIES

01 Ingest concurrent journeys and retain web/mobile context.

02 Reconstruct sessions and expose funnel and abandonment analytics.

03 Show dwell-time distributions and channel ROI from documented inputs.

F MINIMUM ACCEPTANCE CRITERIA

- ☐ All specified event types and anomaly patterns appear in generated data.
- ☐ Session/funnel logic is reproducible and handles documented duplicate cases.
- ☐ The four specified analytical outputs reconcile with serving queries.
- ☐ The shared architecture runs end to end with validation and orchestration evidence.

G EXPECTED DELIVERABLES

Traffic generator and session schema, anomaly controls, streaming jobs, lakehouse/warehouse definitions, funnel/session/ROI queries, BI dashboard and execution evidence.

IMPLEMENTATION RECOMMENDATIONS

Declare synthetic revenue and marketing-cost assumptions needed for ROI. Document attribution and bot-handling rules so metrics remain interpretable. Separate event-time calculations from processing-time latency.

H ADVANCED ENHANCEMENTS (OPTIONAL)

Late-event reconciliation, identity-resolution experiments, live pipeline health monitoring or incremental session recomputation.

The complete submission package

01 **Source code**

Organized repository or source files containing generator, ingestion, Spark, warehouse and orchestration code.

02 **Technical documentation**

README, architecture diagram, dependencies, configuration and complete execution instructions for the chosen environment.

03 **Data assets**

Generator scripts, event schemas, configurations, anomaly controls and representative sample events.

04 **Technical evidence**

Quality results, ingestion/DLQ logs, Spark outputs, table history, SCD demonstrations and workflow logs.

05 **Working solution**

Executable pipeline, transactional lakehouse, dimensional model, serving queries and working BI dashboard.

06 **Presentation deck**

Explain the problem, architecture, implementation, results, limitations and business value.

07 **Demonstration**

Prepare a working live demo. A recorded fallback is recommended where connectivity or runtime could disrupt delivery.

08 **Team contribution summary**

Identify each member’s technical responsibilities and the components or evidence they contributed.

Five readiness gates

01 Project Scoping

Define the problem, core requirements, responsibilities and initial architecture.

Output: Scope statement, objectives and component ownership.

Ready when: The intended outputs and acceptance evidence are clear.

02 Data & Architecture Preparation

Build data schemas and generator scenarios, configure the environment and validate ingestion.

Output: Data or generator assets, environment setup and architecture diagram.

Ready when: A small representative path runs through the planned components.

03 Core Implementation

Implement ingestion, Spark transformations, transactional tables, dimensional serving and BI.

Output: Working core modules and initial integrated outputs.

Ready when: The major functions operate together on representative data.

04 Testing & Refinement

Validate data and KPIs, test failure paths, optimize processing and refine documentation.

Output: Evaluation or test evidence, corrected defects and updated run instructions.

Ready when: Results are traceable and core acceptance criteria are demonstrable.

05 Final Demonstration

Rehearse the complete demo, organize final artifacts and prepare the presentation.

Output: Working solution, final documentation, presentation and contribution summary.

Ready when: The team can reproduce the workflow and explain results and limitations.

FINAL PROJECT READINESS CHECKLIST

Confirm the evidence before the final demonstration.

- ☐ Project objective and stakeholder outcome are clearly defined.
- ☐ Core technical requirements and selected domain scope are complete.
- ☐ All required system components work together.
- ☐ Generated data, processing outputs and analytical KPIs are validated.
- ☐ Major technical decisions and limitations are documented.
- ☐ Outputs are reproducible using the supplied instructions.
- ☐ The final architecture diagram matches the implemented system.
- ☐ The live demonstration has been tested from start to finish.
- ☐ The final presentation explains the work and its business value.
- ☐ Required deliverables are complete and organized.
- ☐ Each team member can explain their technical contribution.

Simulation / Pipelines / Lakehouse / Analytics